# PING PONG EN LENGUAJE ENSAMBLADOR

## DOCUMENTO TÉCNICO - UNIVERSIDAD NACIONAL

**Curso:** Arquitectura de Computadoras
**Profesor:** Cesar Valverde
**Integrantes:** Warner Guevara, Jordana Montoya
**Fecha de entrega:** 29/10/2026

## Índice

1. Introducción
2. Descripción del proyecto
3. Justificación
4. Objetivos
5. Alcance y limitaciones
6. Entorno de desarrollo y fundamentos técnicos
7. Organización del programa
8. Representación de pantalla y memoria de video
9. Funcionamiento del juego
10. Manejo de teclado, tiempo y estados
11. Errores encontrados y soluciones aplicadas
12. Plan de pruebas
13. Uso educativo
14. Conclusiones
15. Recomendaciones

# 1. Introducción

Este documento técnico describe el diseño y el funcionamiento de un videojuego de ping pong para dos jugadores, desarrollado en lenguaje ensamblador para la arquitectura 8086 y preparado para ejecutarse en el emulador emu8086. El informe presenta la estructura del programa, el uso de memoria de video, las variables principales, el control del teclado y del tiempo, la lógica de movimiento y colisiones, así como los errores encontrados durante el desarrollo.

La versión actual incorpora un obstáculo central móvil, paletas de tamaño progresivo y una habilidad de ampliación temporal por jugador. Estas mecánicas se integran con la puntuación, la pausa, la revancha y la velocidad de 5 a 7.

El objetivo de la documentación es explicar tanto el comportamiento visible del juego como los mecanismos de bajo nivel que lo hacen posible. El apartado de pruebas distingue la validación manual reportada por los integrantes y las comprobaciones auxiliares del código.

# 2. Descripción del proyecto

El proyecto implementa una versión de ping pong en la que dos jugadores controlan paletas ubicadas a ambos lados del campo. La pelota se desplaza por el tablero, rebota en los límites superior e inferior y puede ser devuelta por cualquiera de las paletas. Cuando un jugador no logra devolverla, el rival obtiene un punto. La partida termina cuando uno de los jugadores alcanza cinco puntos.

El programa incorpora un menú, controles independientes, pausa, retorno al menú, opción de revancha, marcador y registro del mayor número de devoluciones consecutivas durante la ejecución. El récord se conserva únicamente en memoria mientras el programa permanece abierto; no se guarda en un archivo.

Una barrera roja se mueve verticalmente por el centro y puede devolver la pelota sin intervención del rival. Las paletas normales se reducen de cinco a cuatro y luego a tres celdas según la suma de puntos de ambos jugadores. Cada participante dispone, además, de un único poder para ampliar su paleta a siete celdas durante aproximadamente cinco segundos de juego.

# 3. Justificación

El desarrollo permite aplicar conceptos fundamentales de Arquitectura de Computadoras y programación de bajo nivel. El programa requiere controlar directamente registros, interrupciones, coordenadas de pantalla, memoria de video, flujo de ejecución y temporización.

Además, el proyecto permite estudiar cómo decisiones de implementación, por ejemplo redibujar toda la pantalla o actualizar solamente las posiciones modificadas, influyen en el rendimiento y en la experiencia visual.

Las nuevas mecánicas amplían este aprendizaje: la barrera requiere colisiones adicionales, las alturas variables afectan dibujo y límites, y los poderes necesitan temporizadores y control de usos independientes.

# 4. Objetivos

## 4.1. Objetivo general

Desarrollar y documentar un videojuego de ping pong para dos jugadores en lenguaje ensamblador 8086, aplicando conceptos de manejo de memoria, interrupciones, entrada por teclado, control temporal, procedimientos y lógica de colisiones.

## 4.2. Objetivos específicos

- Implementar el movimiento independiente de las dos paletas mediante controles de teclado.
- Representar el tablero, la pelota, las paletas, el obstáculo y los indicadores en pantalla.
- Controlar el movimiento de la pelota y detectar colisiones con los límites, las paletas y la barrera móvil.
- Administrar la puntuación y finalizar la partida cuando un jugador alcance cinco puntos.
- Incorporar pausa, regreso al menú y opción de revancha.
- Reducir progresivamente las alturas de las paletas y gestionar una ampliación temporal de un uso por jugador.
- Mejorar el rendimiento visual mediante dibujo incremental y actualización selectiva del marcador.
- Verificar el comportamiento mediante comprobaciones auxiliares y pruebas de ejecución en emu8086.

# 5. Alcance y limitaciones

## 5.1. Funcionalidades incluidas

- Menú principal y opciones para iniciar una partida o salir.
- Dos paletas controladas por jugadores diferentes.
- Movimiento de pelota, rebotes, colisiones y puntuación.
- Obstáculo central móvil que invierte la dirección horizontal de la pelota.
- Alturas normales de cinco, cuatro y tres celdas según los puntos totales.
- Un poder por jugador y por partida para ampliar la paleta a siete celdas.
- Pausa y regreso al menú durante una partida.
- Victoria al alcanzar cinco puntos y posibilidad de revancha.
- Aumento escalonado de velocidad de 5 a 7 y conteo de devoluciones consecutivas.
- Registro del récord durante la sesión actual.

### 5.2. Limitaciones

- El récord no persiste después de cerrar el programa.
- La ejecución está orientada a emu8086 y a los servicios BIOS/DOS compatibles.
- La lectura de teclado se realiza por eventos BIOS; la repetición de varias teclas mantenidas depende del teclado y del emulador. No se instala un controlador de pulsación y liberación simultáneas.
- La rapidez efectiva y la duración perceptible de los poderes dependen de la ejecución del emulador.
- Las comprobaciones auxiliares y la revisión manual en emu8086 tienen alcances diferentes. Los estados de la revisión manual se consignan conforme a la validación reportada por los integrantes.

## 6. Entorno de desarrollo y fundamentos técnicos

### 6.1. Arquitectura y formato

El programa está escrito para Intel 8086 y se prepara como ejecutable COM. La directiva #make_COM# selecciona este formato en emu8086. ORG 100h establece el origen habitual, ya que los primeros 256 bytes del segmento se reservan para el Program Segment Prefix (PSP). La instrucción JMP inicio transfiere el control a la etiqueta de inicio, evitando ejecutar los datos iniciales.

### 6.2. Emulador

emu8086 se utiliza como entorno de ensamblado y ejecución. Las pruebas de teclado, pantalla, temporización y rendimiento deben realizarse en este entorno, porque una revisión estática o un emulador auxiliar no demuestra por sí solo el comportamiento dentro de emu8086.

### 6.3. Interrupciones

| Interrupción | Servicio | Uso en el programa |
|---|---|---|
| INT 10h | BIOS Video | Seleccionar modo texto, colocar el cursor y limpiar regiones. |
| INT 16h | BIOS Teclado | Consultar teclas sin bloqueo y leer eventos disponibles. |
| INT 1Ah | BIOS Reloj | Consultar ticks para el movimiento, la barrera y los poderes. |
| INT 21h | DOS | Mostrar cadenas con AH=09h, caracteres con AH=02h y terminar con AH=4Ch. |

La función concreta depende del valor cargado en AH y de los registros de entrada. Por ejemplo, INT 1Ah con AH=00h devuelve el contador en CX:DX; el programa utiliza la palabra baja DX.

### 6.4. Procedimientos y pila

CALL guarda la dirección de retorno y RET la recupera. PUSH y POP preservan registros en orden inverso. El programa utiliza SP y registros 8086; las referencias del enunciado a $sp, $ra, $s0-$s7 y .align corresponden a la alternativa MIPS y no se trasladan literalmente a este código.

Las rutinas superiores consideran los registros generales como temporales. celda preserva AX, BX, DX y DI; pintarPaleta conserva CX y DX; numero conserva AX, BX y DX. Las rutinas de dibujo que cambian ES lo restauran. No se requiere un marco BP cuando no existen parámetros ni variables locales en la pila.

## 7. Organización del programa

El programa se organiza mediante datos, etiquetas, procedimientos y un ciclo principal que coordina entrada, tiempo, actualización de la lógica y dibujo.

### 7.1. Variables principales

| Variable | Descripción |
|---|---|
| puntos / puntos2 | Puntuación del jugador 1 y del jugador 2. |
| record / rally | Mayor racha de la sesión y devoluciones del intercambio actual. |
| velocidad / pasos | Subpasos por actualización y subpasos pendientes. |
| paleta / paleta2 | Primera fila de las paletas izquierda y derecha. |
| pelotaX / pelotaY | Coordenadas actuales de la pelota. |
| dxPelota / dyPelota | Dirección horizontal y vertical: 1 o -1. |
| estado | Ejecución, pausa, solicitud de menú o ganador. |
| ultimoTick / deltaTick | Referencia de reloj y ticks transcurridos. |
| tamBase / tam1 / tam2 | Altura normal y alturas reales de ambas paletas. |
| anteriorTam1 / anteriorTam2 | Alturas previas para borrar completamente los objetos. |
| usado1 / usado2 | Indican si cada jugador ya consumió su poder. |
| poder1 / poder2 | Palabras de 16 bits con los ticks restantes de ampliación. |
| obstaculoY / anteriorObstaculoY | Fila actual y anterior de la barrera. |
| direccionObstaculo | Sentido de movimiento vertical del obstáculo. |
| relojObstaculo | Ticks acumulados para desplazar la barrera. |
| anteriorX / anteriorY | Posición anterior de la pelota. |
| anteriorPaleta / anteriorPaleta2 | Posiciones anteriores de ambas paletas. |

marcadorSucio indica que deben actualizarse los indicadores. Las posiciones y estados se almacenan en bytes mediante DB. Los temporizadores poder1, poder2, ultimoTick, deltaTick y relojObstaculo utilizan palabras mediante DW. Una carga consumida y un efecto activo son conceptos distintos: al expirar poder1 queda en cero, pero usado1 permanece en uno.

### 7.2. Macro de impresión

IMPRIMIR agrupa la presentación de cadenas. LEA DX, cadena obtiene la dirección del mensaje; MOV AH, 09h selecciona la salida de texto de DOS e INT 21h imprime desde DS:DX hasta encontrar el símbolo $. Una macro se expande donde se utiliza y no requiere CALL ni RET.

### 7.3. Procedimientos principales

| Procedimiento | Cometido |
|---|---|
| inicializar | Preparar nueva partida, barrera, alturas y los dos usos de poder. |
| centrarPelota | Reiniciar saque, rally y velocidad, conservando poderes activos. |
| teclado | Procesar movimientos, Q/L, pausa y menú. |
| leerTick / esperarTiempo | Consultar el reloj y calcular deltaTick. |
| actualizar / moverPelota | Actualizar mecánicas y ejecutar movimiento con colisiones unitarias. |
| tocarPaleta | Evaluar contacto con la altura real y devolver el resultado mediante CF. |
| ajustarTamanos | Calcular base, aplicar ampliaciones y limitar posiciones. |
| activarPoder1 / activarPoder2 | Consumir una carga y activar 91 ticks de efecto. |
| actualizarExtras | Descontar poderes y mover la barrera. |
| descontarPoderes | Finalizar efectos y recuperar la altura normal vigente. |
| puntoJ1 / puntoJ2 | Sumar puntos, ajustar alturas y comprobar victoria. |
| dibujar / pintarPaleta / celda | Actualizar objetos según posiciones y dimensiones reales. |
| marcador / mostrarExtras | Mostrar puntos, récord, rally, velocidad, alturas y poderes. |
| limpiar / dibujarBordes | Limpiar pantallas y dibujar el campo al inicio. |
| mostrarPausa | Gestionar el mensaje de pausa. |
| numero | Presentar valores de 0 a 99 como dos dígitos ASCII. |

# 8. Representación de pantalla y memoria de video

## 8.1. Coordenadas y desplazamiento

En el modo texto de 80 columnas, cada celda ocupa dos bytes: carácter y atributo de color. Con índices de fila y columna que empiezan en cero:

$$\text{desplazamiento} = 2 \times (\text{fila} \times 80 + \text{columna})$$

Ejemplo práctico para la fila 12 y la columna 32:

$$2 \times (12 \times 80 + 32) = 1984 \text{ bytes} = 07C0h$$

La celda correspondiente se localiza en B800:07C0h.

## 8.2. Caracteres y atributos

| Objeto | Palabra | Carácter y color |
|---|---|---|
| Pelota | 0E4Fh | 4Fh = O; atributo 0Eh = amarillo sobre negro. |
| Paleta J1 | 0A7Ch | 7Ch = \|; atributo 0Ah = verde claro. |
| Paleta J2 | 0D7Ch | 7Ch = \|; atributo 0Dh = magenta claro. |
| Obstáculo | 0C23h | 23h = #; atributo 0Ch = rojo claro. |

## 8.3. Dibujo incremental

- Borrar la posición anterior de la pelota y de las paletas mediante espacios.
- Borrar la barrera en su posición previa y repintarla en la actual.
- Usar las alturas anteriores para borrar completas las paletas al reducirse o expirar un poder.
- Dibujar la barrera, las paletas y por último la pelota para restaurar celdas compartidas.
- Actualizar el marcador y los estados únicamente cuando cambian sus valores.
- Dibujar los bordes al iniciar la partida, no en cada cuadro.

## 8.4. Distribución de la interfaz

La fila 0 contiene puntos, récord, rally y velocidad. La fila 1 muestra TAM J1/J2 y los estados Q J1 y L J2. Los bordes están en las filas 2 y 22; el área jugable comprende las filas 3 a 21. La barrera ocupa cuatro filas en la columna 33. La pausa aparece en la fila 23 y las instrucciones en la 24.

# 9. Funcionamiento del juego

## 9.1. Estados del programa

- 0: partida en ejecución.
- 1: pausa.
- 2: solicitud de regreso al menú.
- 3: victoria del jugador 1 y derrota del jugador 2.
- 4: victoria del jugador 2 y derrota del jugador 1.

## 9.2. Ciclo principal

1. Atender las entradas del teclado.
2. Comprobar pausa o regreso al menú.
3. Esperar el siguiente tick y calcular el tiempo transcurrido.
4. Actualizar duración de poderes y movimiento del obstáculo.
5. Ejecutar los subpasos de la pelota y verificar colisiones.
6. Actualizar la pantalla mediante dibujo incremental.
7. Comprobar si la partida terminó.

## 9.3. Movimiento y límites de la pelota

La pelota utiliza pelotaX y pelotaY y los incrementos dxPelota y dyPelota. Se mueve entre las filas 3 y 21. Cuando intenta superar esos límites se invierte la dirección vertical para producir el rebote. Cada actualización se divide en avances unitarios, de modo que una velocidad mayor no omita una paleta ni la barrera.

## 9.4. Colisiones con las paletas

La colisión izquierda se evalúa en la columna 3 y la derecha en la 62. La condición anterior de altura fija se sustituye por una condición basada en la altura real:

```
filaPaleta ≤ pelotaY < filaPaleta + altura
```

La altura corresponde a tam1 o tam2 y puede valer 3, 4, 5 o 7. tocarPaleta recibe la primera fila en BL y la altura en CL. Calcula la posición relativa de la pelota y compara con CL; también adapta el punto medio de golpe a la altura actual.

- CLC indica contacto con la paleta.
- STC indica fallo o ausencia de contacto.
- JNC permite seleccionar el rebote cuando CF está desactivado.

El golpe superior orienta la pelota hacia arriba, el inferior hacia abajo y el centro conserva la dirección vertical. Cuando se registra un punto se cancelan los subpasos restantes y se prepara un nuevo saque.

## 9.5. Puntuación y victoria

Cada jugador tiene una puntuación independiente. Si falla J1, suma J2; si falla J2, suma J1. La partida termina al alcanzar cinco puntos. Después de un punto se recalculan las alturas, se reinician rally y velocidad y la pelota sale inicialmente hacia quien falló. El obstáculo puede interceptar ese saque.

### 9.6. Aumento de velocidad y récord

- Velocidad base: 5.
- Tras dos devoluciones consecutivas: 6.
- Tras cuatro devoluciones consecutivas: 7.

VEL representa subpasos por actualización, no cuadros por segundo. El aumento se aplica al siguiente cuadro. record conserva la mayor racha durante la sesión; rally cuenta solo contactos con paletas, no rebotes de la barrera. Los dos contadores se limitan a 99.

### 9.7. Obstáculo central móvil

Una barrera roja de cuatro celdas se desplaza verticalmente por la columna 33. Su primera fila varía de 3 a 18 para que su última celda no supere la fila 21. Se mueve una fila cada dos ticks de juego y cambia de dirección en los límites.

En cada subpaso se comprueba si la pelota llega a esa columna dentro de las cuatro filas ocupadas. Si coincide, se invierte dxPelota; si no, la pelota continúa. El contacto no otorga puntos ni aumenta el rally.

La barrera se mueve antes de los subpasos del cuadro. La colisión se evalúa al cruzar la columna; no se utiliza física continua de superficies. Se limita la recuperación tras retrasos prolongados para evitar bucles excesivos.

### 9.8. Reducción progresiva de las paletas

La altura normal depende de la suma de puntos de ambos jugadores:

| Puntos totales | Altura normal |
|---|---|
| 0 o 1 | 5 celdas |
| 2 o 3 | 4 celdas |
| 4 o más | 3 celdas |

Con 1-1, por ejemplo, las paletas normales miden cuatro celdas; con 2-2, tres. ajustarTamanos aplica el cambio después de cada punto. La primera fila se limita al intervalo entre 3 y 22 menos la altura real. Si una ampliación está activa, la paleta correspondiente mantiene siete celdas hasta que termine.

### 9.9. Habilidad especial por jugador

Q activa la ampliación de J1 y L la de J2. Cada participante dispone de un uso por partida completa. El efecto asigna siete celdas durante 91 ticks de juego, aproximadamente cinco segundos en un reloj BIOS normal.

Durante la pausa no se descuenta tiempo. Un punto no cancela un efecto activo ni recarga su uso. Repetir la tecla no aumenta la duración. Al expirar, la paleta vuelve a la altura normal que corresponda al marcador actual. Una revancha o nueva partida restablece ambas cargas.

| Estado visible | Significado |
|---|---|
| LISTO | La habilidad está disponible. |
| ACTIVO | La paleta tiene siete celdas temporalmente. |
| USADO | La carga ya se consumió y el efecto terminó. |

# 10. Manejo de teclado, tiempo y estados

## 10.1. Controles del juego

| Tecla o control | Acción |
|---|---|
| W / S | Subir o bajar la paleta de J1. |
| Flecha arriba / abajo | Subir o bajar la paleta de J2. |
| Q | Activar la ampliación de J1, si está LISTO. |
| L | Activar la ampliación de J2, si está LISTO. |
| P | Pausar o reanudar. |
| ESC durante la partida | Solicitar el menú. |
| R en el resultado | Iniciar revancha y recargar los dos poderes. |
| M en el resultado | Regresar al menú. |
| ESC en el resultado | Salir. |
| 1 en menú / 2 o ESC en menú | Iniciar partida / salir. |

Las letras se aceptan también en mayúscula. Los movimientos y las activaciones se ignoran durante la pausa; se siguen atendiendo P y ESC.

## 10.2. Control temporal

INT 1Ah proporciona el contador BIOS. esperarTiempo compara con ultimoTick y guarda el intervalo en deltaTick. La resta de la palabra baja admite su vuelta de 65535 a 0. El avance se solicita por tick, pero la cadencia efectiva depende del rendimiento de emu8086.

descontarPoderes compara el intervalo con la duración restante antes de restar. Si lo iguala o supera, asigna cero y recalcula la altura; así evita que la resta desborde y prolongue el efecto. relojObstaculo acumula ticks para mover la barrera una fila cada dos.

## 10.3. Pausa y reinicio

actualizar solo procesa objetos y temporizadores con estado 0. Al reanudar se toma una nueva referencia temporal para no consumir el intervalo de pausa. El mensaje se muestra en la fila 23 sin ocultar tamaños ni estados de poder.

La revancha reinicia marcadores, alturas, barrera y cargas. El récord permanece en memoria durante toda la ejecución.

## 11. Errores encontrados y soluciones aplicadas

- **La pelota permanecía inmóvil:** se cambió la consulta directa de memoria por INT 1Ah para sincronizar el movimiento. Posteriormente el usuario confirmó que la pelota se movía.
- **Parpadeo de pantalla:** se sustituyó el redibujado total por una actualización selectiva de posiciones. El usuario confirmó la eliminación del parpadeo.
- **Lentitud durante la partida:** se redujo la espera y se eliminaron la reconstrucción del borde y las actualizaciones innecesarias del marcador.
- **Alcance de un salto condicional:** en una verificación auxiliar anterior se sustituyó el salto fuera de rango por una condición inversa seguida de JMP.
- **Ampliación a dos jugadores:** se añadieron paleta2, controles independientes y puntuación doble. Fue una mejora de diseño, no un error de ejecución.
- **Velocidad y atravesamiento:** se dividió el movimiento en subpasos que comprueban contactos en cada avance.

### 11.1. Casos prevenidos con las nuevas mecánicas

Los siguientes casos se cubrieron en las comprobaciones auxiliares; no se presentan como errores observados por el usuario dentro de emu8086.

- **Restos al reducir una paleta:** se guardan anteriorTam1 y anteriorTam2 para borrar toda la altura previa antes de dibujar la actual.
- **Paleta fuera del campo al ampliar:** se limita la primera fila utilizando la altura real, incluida la de siete celdas.
- **Temporizador que desborda al expirar:** se compara antes de restar y se satura en cero.
- **Poder reutilizado o extendido:** usado1 y usado2 separan la carga consumida de la duración restante y evitan una nueva activación.
- **Poder consumido durante la pausa:** se detienen las actualizaciones y se renueva la referencia temporal al continuar.
- **Mensaje de pausa que oculta los poderes:** se trasladó a la fila 23.

## 12. Plan de pruebas

El plan conserva los casos del documento original y añade los relacionados con obstáculo, alturas y poderes. Según la validación reportada por los integrantes, todos los casos manuales se registran como verificados y correctos en emu8086.

### 12.1. Pruebas en emu8086

| Caso | Verificación | Estado |
|---|---|---|
| Compilación | Ensamblar y ejecutar el ASM original sin errores. | Verificado y correcto. |
| Controles | W/S y flechas mueven sus respectivas paletas. | Verificado y correcto. |
| Límites | Probar alturas 3, 4, 5 y 7 sin salir del tablero. | Verificado y correcto. |
| Rebotes | Comprobar los bordes superior e inferior. | Verificado y correcto. |
| Colisiones | Devolver con ambas paletas y con sus diferentes alturas. | Verificado y correcto. |
| Puntuación | Cada fallo suma un punto al rival. | Verificado y correcto. |
| Aceleración | VEL pasa de 5 a 6 y 7 según el rally. | Verificado y correcto. |
| Obstáculo | Movimiento, límites, rebotes por ambos lados y paso por los huecos. | Verificado y correcto. |
| Reducción | Altura normal 5, 4 y 3 con los puntos totales indicados. | Verificado y correcto. |
| Poderes Q y L | Activación independiente, siete celdas y expiración aproximada. | Verificado y correcto. |
| Uso único | Repetir Q/L no prolonga ni recarga la habilidad. | Verificado y correcto. |
| Pausa | Congela pelota, barrera y duración de poderes. | Verificado y correcto. |
| Victoria | Identificar al ganador al llegar a cinco puntos. | Verificado y correcto. |
| Revancha | Reiniciar marcadores y recuperar ambas cargas con R. | Verificado y correcto. |
| Récord | Conservar la máxima racha durante la sesión. | Verificado y correcto. |
| Rendimiento y dibujo | Sin retrasos severos, restos de caracteres ni parpadeo molesto. | Verificado y correcto. |

### 12.2. Comprobaciones auxiliares realizadas

Se tradujo mecánicamente la sintaxis del ASM a GNU as en modo de 16 bits, se ensambló y enlazó el resultado y se ejecutaron las instrucciones con Unicorn. Los servicios de video, teclado, reloj y DOS fueron simulados por un arnés de pruebas. Esto permite comprobar lógica, memoria de video y pila, pero no equivale a compilar o jugar dentro de emu8086.

Pasaron 13 grupos de comprobaciones:

| Grupo | Resultado |
|---|---|
| Inicialización, tablero y estados de poderes | Aprobado. |
| Reducción con dos y cuatro puntos totales | Aprobado. |
| Poderes independientes, duración de 91 ticks y uso único | Aprobado. |
| Pausa, reanudación y conservación del tiempo restante | Aprobado. |
| Poder entre puntos y recarga en revancha | Aprobado. |
| Límites de ambas paletas con alturas 3, 4, 5 y 7 | Aprobado. |
| Barrera: contactos por ambos lados, paso y límites | Aprobado. |
| Contactos y fallos con todas las alturas | Aprobado. |
| Velocidad de 5 a 7 y detección por subpasos | Aprobado. |
| Dibujo sin restos e indicadores correctos | Aprobado. |
| Victoria de ambos jugadores y cancelación del cuadro tras punto | Aprobado. |
| Estabilidad con controles, poderes y pila equilibrada | 1500 actualizaciones. |
| Flujo principal con dos partidas, revancha, menú y salida | 270 iteraciones; salida correcta. |

El equilibrio de SP se comprobó al regresar de las llamadas. Los datos y el método de esta verificación se documentan también en pruebas/RESULTADOS.md del proyecto.

### 12.3. Evidencias por incorporar

Agregar capturas reales de la interfaz, la barrera, una paleta ampliada, la reducción de tamaños, la pausa y el resultado final. Registrar el resultado obtenido y cualquier diferencia observada en el emulador, sin reemplazar estos casos por evidencias simuladas.

## 13. Uso educativo

El proyecto integra elementos de programación de bajo nivel en una aplicación interactiva: teclado, tiempo, caracteres en pantalla, coordenadas, colisiones y estados. Las nuevas mecánicas añaden temporizadores, tamaños variables y control de recursos de uso limitado.

Su desarrollo y documentación tienen fines educativos dentro del curso Arquitectura de Computadoras. La revisión del código y las pruebas permiten comprender las decisiones de implementación y sus límites.

## 14. Conclusiones

El desarrollo del juego demuestra cómo la coordinación de rutinas simples permite construir software interactivo. La gestión explícita de la memoria de video y las interrupciones BIOS es fundamental para el rendimiento de una aplicación 8086 y relaciona el diseño del software con el entorno de ejecución.

La barrera móvil introduce devoluciones imprevistas; las paletas más pequeñas aumentan la precisión requerida; y la habilidad de ampliación incorpora una decisión estratégica de uso limitado. Las tres funciones se integran con las reglas de victoria, la aceleración, la pausa y la revancha.

Las comprobaciones auxiliares respaldan la lógica de esta revisión. La validación manual reportada por los integrantes registra todos los casos como verificados y correctos en emu8086. El récord de sesión y la lectura por eventos BIOS permanecen como limitaciones documentadas.

## 15. Recomendaciones

- Conservar el registro de las pruebas verificadas y repetir los casos afectados cuando se modifique el código.
- Mantener identificadores, controles y comentarios del código alineados con esta documentación.
- Verificar que los saltos condicionales mantengan rangos relativos válidos para 8086.
- Probar situaciones límite de colisión a altas velocidades, incluidos los dos lados de la barrera.
- Comprobar la reducción, ampliación y expiración cerca de los bordes y durante una pausa.
- Adjuntar capturas reales de emu8086 como evidencia.
- Conservar el repositorio Git y las contribuciones reales de los integrantes exigidas por el proyecto.
- Considerar la persistencia del récord en un archivo mediante INT 21h para una futura revisión.